



SOICT

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

IT6390E — KNOWLEDGE GRAPHS
Capstone Project Report

Movies Linked Open Data: A Five-Star Knowledge Graph of 250 Films Linked to Wikidata and DBpedia

Group 2

Đỗ Tuấn Minh
Nguyễn Tiến Đức
Hoàng Huy Chiến

Source code and data: <https://github.com/minhdo2207/it6390e-lod-capstone>

Hanoi, October 2026

Abstract

This report presents a Linked Open Data application in the movie domain that reaches all five stars of the deployment scheme proposed by Berners-Lee. We designed a small OWL ontology that reuses schema.org, FOAF, the DBpedia ontology and Dublin Core, collected the 250 best-rated films of the TMDb 5000 dataset, converted them to RDF with stable URIs, and linked films and people to Wikidata and DBpedia. Linking matches on the TMDb identifiers that Wikidata already stores instead of on titles, which produced 920 `owl:sameAs` links to Wikidata and 912 to DBpedia. A manual evaluation on a random sample of 30 resources measured a precision of 100% for Wikidata and 96.7% for DBpedia; the single error revealed that DBpedia redirect resources carry stale `owl:sameAs` statements, which we then excluded. Defined classes and one SWRL rule allow the HermiT reasoner to infer 29 action films, 16 comedies, 32 award-winning directors and 5 actor-directors, and a deliberately contradictory record is detected as an inconsistency. The resulting graph of 12,013 triples is queried with 21 SPARQL queries, ten of which are federated queries that retrieve facts from Wikidata and DBpedia at query time.

Keywords: Linked Open Data, ontology engineering, OWL, SWRL, entity linking, SPARQL federation.

Contents

1	Introduction	2
2	Ontology Design	2
2.1	Method and competency questions	2
2.2	Classes and properties	3
2.3	Defined classes and a SWRL rule	4
2.4	Cast roles as an n-ary relation	4
2.5	A modelling trade-off	4
3	Data Collection	4
4	Conversion to Four-Star RDF	5
4.1	Pipeline	5
4.2	URI design and mapping	5
5	Linking to Wikidata and DBpedia	6
5.1	A failed baseline: matching on title and year	6
5.2	Identifier-based matching	6
5.3	Evaluation	7
5.4	Failure analysis: redirect resources	7
6	SPARQL Query Interface	7
7	Reasoning Evaluation	8
8	Dataset Metadata	10
9	Discussion and Limitations	10

10 Conclusion and Future Work	11
Team Contributions	11

1 Introduction

Linked Open Data (LOD) publishes structured data on the Web so that it can be combined with other datasets: entities are named by HTTP URIs, described in RDF, and connected to entities in other datasets [2]. Berners-Lee’s five-star scheme grades how far a dataset goes on this path [1]: openly licensed data on the Web (one star), machine-readable structure (two), a non-proprietary format (three), W3C standards with URIs for things (four), and links to other datasets (five).

The capstone asks for a LOD application in a chosen domain, built in five steps: (1) define an ontology, (2) collect data, (3) convert the data to four-star data, (4) link it to other datasets to reach five stars, and (5) provide a SPARQL query interface. We chose the movie domain for two reasons. Open movie data is plentiful, and the two central hubs of the LOD cloud, DBpedia [3] and Wikidata [4], describe films, directors and actors in depth, which makes the fifth star reachable within the project time.

The contributions of this work are:

- an ontology with nine core classes, defined classes and a SWRL rule, plus an n-ary model of cast roles (Section 2);
- a reproducible pipeline from the TMDb 5000 dataset to linked RDF (Sections 3–4);
- an identifier-based linking method with a manual precision evaluation and an analysis of a failure mode of DBpedia links (Section 5);
- a query interface with local, n-ary and federated SPARQL queries, and a reasoning evaluation in Protégé (Sections 6–7).

Table 1 summarises how the project meets each level of the scheme.

Table 1: The five-star scheme applied to this project.

Stars	Requirement	This project
★	On the Web under an open licence	Public repository; RDF under CC BY-NC 4.0, code under MIT
★★	Machine-readable structured data	Cleaned dataset as a CSV table
★★★	Non-proprietary format	CSV and Turtle
★★★★	URIs and W3C standards	A URI for every film, person, genre, studio, country and cast role; RDF, OWL, SPARQL
★★★★★	Links to other data	920 owl:sameAs links to Wikidata, 912 to DBpedia

2 Ontology Design

2.1 Method and competency questions

We followed a lightweight version of Methontology [5]: specification through competency questions, conceptualisation as a glossary of terms, reuse of existing vocabularies, then implementation in OWL 2 with Protégé. Competency questions, as proposed by Grüninger and Fox [6], were fixed before any modelling. They determined which classes and properties were needed, and each was later implemented as a SPARQL query (Section 6):

1. Who directed movie X ? (query 1)
2. Which movies were released after year Y ? (query 2)

3. Which director has the most movies? (query 3)
4. Which movies feature actor Y? (query 5)
5. Which character did an actor play, and how high was the actor billed? (queries 16–17)

2.2 Classes and properties

Following LOD best practice, existing vocabulary terms were reused before new ones were minted [2]: `schema:Movie`, `foaf:Person`, `dbo:director`, `dbo:starring`, `dc:title` and `foaf:name`. Local terms in the namespace `http://it6390e-group2.example.org/ontology/` (prefix `ont:`) cover what these vocabularies lack. Figure 1 shows the core structure and Table 2 lists the terms.

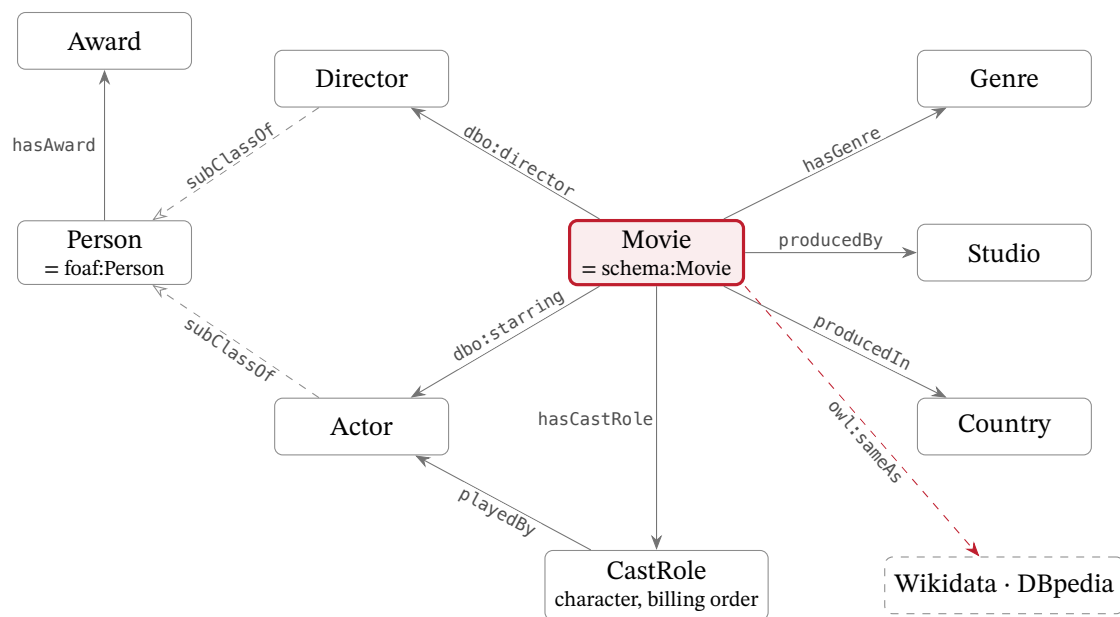


Figure 1: Core classes and properties of the ontology. Dashed open arrows denote `rdfs:subClassOf`; the dashed red arrow denotes the `owl:sameAs` links to external datasets.

Table 2: Main terms of the ontology (ontology/movies.ttl).

Term	Kind	Definition / source
Movie	class	equivalent to <code>schema:Movie</code>
Person	class	equivalent to <code>foaf:Person</code>
Director, Actor	class	subclasses of Person
Genre, Studio, Country, Award	class	local classes
CastRole	class	one actor appearing in one movie (n-ary relation)
<code>dbo:director</code> , <code>dbo:starring</code>	object property	Movie → Director / Actor (DBpedia ontology)
<code>hasGenre</code> , <code>producedBy</code> , <code>producedIn</code>	object property	Movie → Genre / Studio / Country
<code>hasAward</code>	object property	Person → Award
<code>hasCastRole</code> , <code>roleInMovie</code> , <code>playedBy</code>	object property	Movie ↔ CastRole (inverses), CastRole → Actor (functional)
<code>releaseYear</code>	datatype property	Movie → <code>xsd:integer</code>
<code>characterName</code> , <code>billingOrder</code>	datatype property	character played, position in the credits

releaseYear uses `xsd:integer` rather than `xsd:gYear` because Hermit does not support `xsd:gYear` in its datatype map.

2.3 Defined classes and a SWRL rule

Three classes are defined by necessary and sufficient conditions, so that their members are inferred rather than asserted. In Manchester syntax:

ActionMovie	EquivalentTo	hasGenre value genre_action
ComedyMovie	EquivalentTo	hasGenre value genre_comedy
ComedyMovie	DisjointWith	ActionMovie
AwardWinningDirector	EquivalentTo	Director and (hasAward some Award)

The first two follow the pattern of `VegetarianPizza` in the Protégé pizza tutorial. The class `ActorDirector`, a person who directed and starred in the same movie, cannot be expressed as an OWL 2 class expression: it requires the same individual to fill two different properties of one movie, and OWL property chains can only derive a property, not test a join of this kind. It is therefore derived by a SWRL rule [8]:

```
Movie(?m) ^ dbo:director(?m, ?p) ^ dbo:starring(?m, ?p) -> ActorDirector(?p)
```

2.4 Cast roles as an n-ary relation

The binary property `dbo:starring` links a movie to an actor but cannot record the character played or the billing position. Following the W3C note on n-ary relations [9], each appearance of an actor in a movie is reified as a `CastRole` individual carrying `characterName` and `billingOrder`. `dbo:starring` is retained for compatibility, and the property chain axiom `hasCastRole o playedBy  $\sqsubseteq$  dbo:starring` lets a reasoner derive it from the roles, so that the two representations cannot diverge. A consistency query confirms that both describe the same 750 actor–movie pairs.

2.5 A modelling trade-off

Under `OntoClean` [7], `Director` and `Actor` are anti-rigid roles rather than rigid types: being a director is not essential to a person. Modelling them as subclasses of `Person` therefore violates the rigidity constraint, and a cleaner design would attach roles to persons through a property such as `hasRole`. We kept the subclass design to limit the scope of the capstone; for actors, the `CastRole` node already provides a role-based model.

3 Data Collection

The data source is the Kaggle “TMDB 5000 Movie Dataset” [16], two CSV files generated from the TMDB API that describe 4,803 films and their cast and crew. The script `prepare_movies_csv.py` keeps films with at least 1,000 votes, sorts them by average rating and selects the first 250, discarding rows without a title, release date or director. For each film it keeps the TMDB identifier, title, release year, runtime, director, the primary genre, and the three first-billed actors with their characters, together with the first production company and country.

The resulting dataset covers films released between 1939 and 2016 and contains 250 films, 672 distinct people (156 directors and 522 actors), 17 genres, 99 studios and 16 countries, with no empty cells. Figure 2 shows the distribution of primary genres. Only the primary genre is converted to RDF; Section 7 explains why.

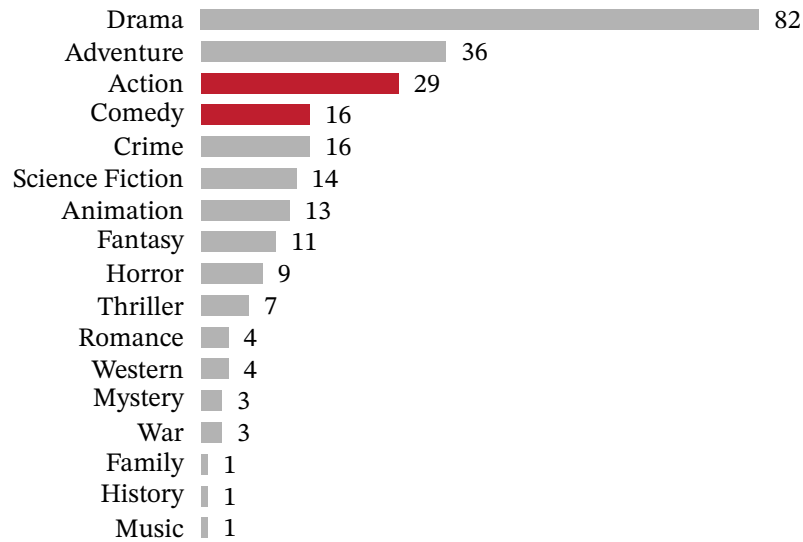


Figure 2: Films per primary genre (250 films). Red: the two genres behind the defined classes `ActionMovie` and `ComedyMovie`.

The data comes from the TMDb API and is not endorsed by TMDb. TMDb's terms restrict commercial use, so the derived RDF is published under CC BY-NC 4.0 with attribution.

4 Conversion to Four-Star RDF

4.1 Pipeline

Four Python scripts, based on `RDFLib`, turn the source files into linked Turtle; every step can be rerun from scratch:

1. `prepare_movies_csv.py` selects the films and writes `movies.csv`;
2. `csv_to_rdf.py` converts each row to RDF and writes `movies.ttl`;
3. `link_wikidata_dbpedia.py` adds the `owl:sameAs` links and a link report (Section 5);
4. `add_awards.py` adds Academy Award for Best Director wins from Wikidata (32 people).

4.2 URI design and mapping

Resources are named under `http://it6390e-group2.example.org/: resource/movie/{tmdb id}`, `resource/person/{tmdb id}` and `resource/role/{movie id}-{actor id}`; genres, studios and countries are named after slugs of their labels. Reusing TMDb identifiers keeps the URIs stable across runs and, as Section 5 shows, provides the key for linking. Genre individuals are named `genre_action`, `genre_comedy`, etc., so that they coincide with the individuals used in the `hasValue` restrictions. Table 3 gives the mapping and Listing 1 one resulting resource.

Table 3: Mapping from CSV columns to RDF.

CSV column	RDF	Vocabulary
id	ex:movie/{id} a schema:Movie	schema.org
title	dc:title	Dublin Core
year	ont:releaseYear (xsd:integer)	local
director_id,	dbo:director ex:person/{id}; foaf:name	DBpedia, FOAF
director_name		
cast_ids, cast_names,	dbo:starring; ont:CastRole with characterName,	DBpedia, local
cast_characters	billingOrder	
genre	ont:hasGenre ont:genre_{name}	local
studio, country	ont:producedBy, ont:producedIn	local

```

ex:movie/1124 a schema:Movie ;
  dc:title "The Prestige" ; ont:releaseYear 2006 ;
  dbo:director ex:person/525 ;
  dbo:starring ex:person/3894, ex:person/3895, ex:person/6968 ;
  ont:hasCastRole ex:role/1124-3894, ex:role/1124-3895, ex:role/1124-6968 ;
  ont:hasGenre ont:genre_drama ;
  ont:producedBy ont:studio/warner-bros ;
  ont:producedIn ont:country/united-states-of-america ;
  owl:sameAs dbr:The_Prestige_(film), wd:Q46551 .

```

Listing 1: The film *The Prestige* in movies_linked.ttl (URIs shortened).

The linked data contains 11,868 triples; together with the ontology and the awards, the graph has 12,013 triples describing 250 films, 672 people and 750 cast roles. Data in RDF, serialised in Turtle, with a URI for every entity, satisfies the fourth star.

5 Linking to Wikidata and DBpedia

5.1 A failed baseline: matching on title and year

The obvious approach is to look up each film on DBpedia by its label and release year. On a sample of eight films this matched none. Inspection showed that DBpedia film resources carry no `dbo:releaseDate`, so the year filter never succeeds; titles are in any case ambiguous because of re-makes and sequels.

5.2 Identifier-based matching

Wikidata records the TMDb identifier of films (property P4947) and of people (P4985). Since our URIs end in that identifier, linking becomes an exact lookup:

1. For batches of 100 identifiers, one SPARQL query to the Wikidata Query Service retrieves the items carrying them. A link is added only if exactly one item carries the identifier.
2. The DBpedia resource is taken from DBpedia's own `owl:sameAs` statements pointing at the Wikidata item, restricted to resources typed `dbo:Film` or `dbo:Person`. This restriction is needed because these statements are noisy: the item for one film also had a soundtrack album and unrelated pages attached. Among the candidates, the resource named after the item's English Wikipedia article is chosen.
3. Ambiguous cases are left unlinked rather than guessed. Two occurred: *Apocalypse Now* shares its TMDb identifier with the *Redux* cut, and the actor J. K. Simmons has a duplicate Wikidata

item.

The result is 920 links to Wikidata (249 of 250 films, 671 of 672 people) and 912 to DBpedia (249 films, 663 people).

5.3 Evaluation

To estimate precision we drew a reproducible random sample of 15 films and 15 people (fixed seed 42) and checked every linked Wikidata and DBpedia page by hand, comparing release year and director for films, and occupation and date of birth for people. Table 4 reports the results.

Table 4: Precision of the links on a random sample of 30 resources.

Target	Correct links	Precision
Wikidata	30 / 30	100%
DBpedia	29 / 30	96.7%

5.4 Failure analysis: redirect resources

The only wrong link, `dbr:Alakina_Mann`, was a redirect resource: the Wikipedia article about the actress had been merged into the article about the film *The Others*, while DBpedia kept the old resource’s type and `owl:sameAs` statement. A scan of all 916 DBpedia links at that time found seven redirect resources, four of which led to a different entity (Alakina Mann → the film *The Others*; Joel Coen → Coen brothers; Anthony Russo → Russo brothers; Lilly Wachowski → The Wachowskis). Such links assert identity between distinct entities, the misuse of `owl:sameAs` documented by Halpin et al. [10]. The linking script now skips redirect resources and follows a redirect only when its target is the resource named after the item’s English article, that is, when the redirect is a plain rename. After this correction none of the 912 DBpedia links is a redirect.

6 SPARQL Query Interface

The graph is queried through two interfaces: `sparql_cli.py`, a terminal tool over RDFLib that runs stored or ad-hoc queries, and the Snap SPARQL Query plugin inside Protégé (the SPARQL plugin bundled with Protégé 5.6.9 fails to load because of a missing library). Snap SPARQL evaluates queries through the active reasoner, so it also returns inferred facts: asking for the instances of `ont:ActionMovie` returns the same 87 IRIs as the DL query in Section 7. Twenty-one queries are provided (Table 5).

Table 5: Query sets.

Queries	File	Purpose
1–5 (local)	<code>sample_queries.rq</code>	competency questions; films linked to DBpedia
16–21 (cast roles)	<code>castrole_queries.rq</code>	characters and billing order; agreement between roles and <code>dbo:starring</code>
6–15 (federated)	<code>federated_queries.rq</code>	facts retrieved from Wikidata and DBpedia with <code>SERVICE</code>

Competency question 3. The query below ranks directors by number of films. Steven Spielberg leads with 8 films, followed by Quentin Tarantino and Christopher Nolan with 7, and Martin Scorsese and David Fincher with 6.

```

SELECT ?directorName (COUNT(?movie) AS ?movieCount) WHERE {
  ?movie dbo:director ?d .
  ?d foaf:name ?directorName .
}
GROUP BY ?directorName ORDER BY DESC(?movieCount) LIMIT 5

```

Cast roles. Query 16 lists the cast of a film through `CastRole`. For *The Dark Knight* it returns Christian Bale as Bruce Wayne (billed first), Heath Ledger as the Joker and Aaron Eckhart as Harvey Dent, information that the binary `dbo:starring` link cannot hold.

Federated queries. Our data stores no dates of birth. Query 6 follows a director's `owl:sameAs` link and retrieves the date and place of birth from Wikidata with a `SERVICE` clause, then joins them with the director's films in our graph. For Christopher Nolan it returns 30 July 1970, London, with seven films from *Memento* (2000) to *Interstellar* (2014). A simplified form:

```

SELECT ?title ?year ?birthDate WHERE {
  ?d foaf:name "Christopher Nolan" ; owl:sameAs ?wd .
  FILTER(STRSTARTS(STR(?wd), "http://www.wikidata.org/entity/"))
  ?movie dbo:director ?d ; dc:title ?title ; ont:releaseYear ?year .
  SERVICE <https://query.wikidata.org/sparql> { ?wd wdt:P569 ?birthDate . }
}

```

Query 7 applies the same idea to every director with at least six films and computes the age at their earliest film in our data (for example, 29 for Steven Spielberg, born 1946, with *Jaws* in 1975). These queries are only possible because of the fifth-star links.

7 Reasoning Evaluation

No film or person in the data is asserted to belong to a defined class. We evaluated the reasoning in two ways: with Hermit [11] run from a script (`check_reasoning.py`, through `owlready2`) and interactively in Protégé 5.6.9 [12]. For Protégé, `make_protege_demo.py` merges the ontology, the linked data and the awards into a single file, since Protégé reasons over one ontology at a time. Classification takes Hermit about three minutes on this file, most of it spent on computing the instances of all object properties.

Table 6: Inferred class members. Protégé lists every IRI of an individual, so linked individuals appear up to three times.

Class	Our resources	Names listed in Protégé
ActionMovie	29 films	87
ComedyMovie	16 films	48
AwardWinningDirector	32 directors	95
ActorDirector (with the SWRL rule)	5 people	15
CastRole	750 roles	750

Both runs give the same counts (Table 6). The larger numbers in Protégé are a direct effect of linking: `owl:sameAs` makes each film or person the same individual as its Wikidata and DBpedia IRIs, and Protégé lists all of them (one director has no DBpedia link, hence 95 rather than 96). The SWRL rule classifies Clint Eastwood, Kevin Costner, Mel Gibson, Terry Gilliam and Woody Allen as actor-directors. Figures 3 and 4 show two of these results.

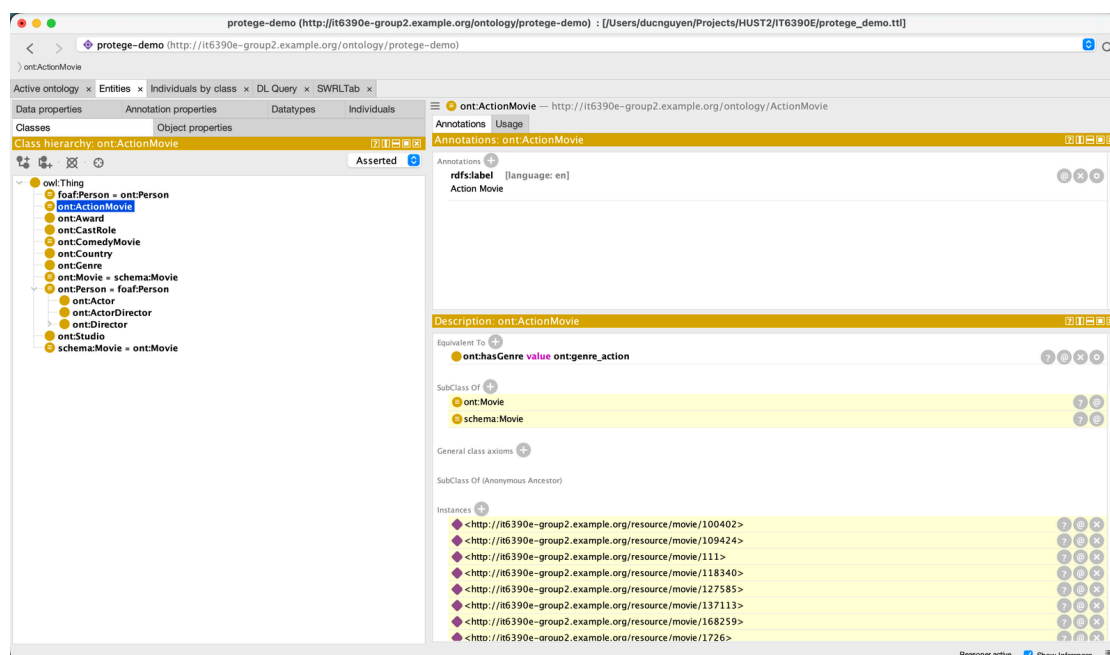


Figure 3: ont:ActionMovie in Protégé: its definition hasGenre value genre_action and its instances inferred by HermiT (highlighted rows).

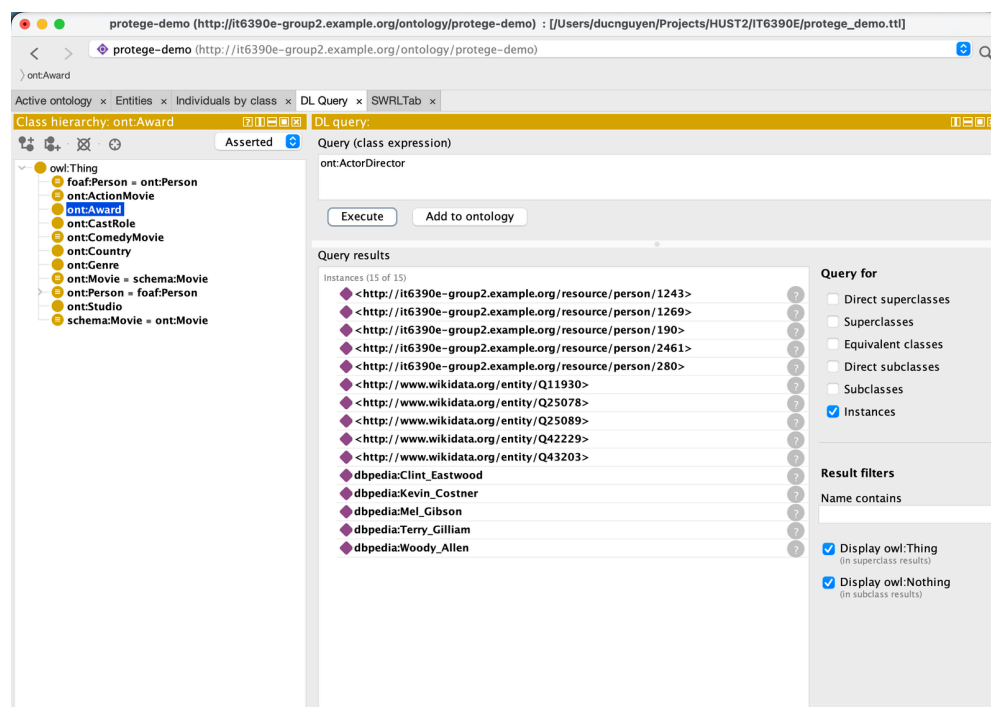


Figure 4: DL query ont:ActorDirector after adding the SWRL rule: five people, each listed under three IRIs.

Inconsistency detection. To test the disjointness axiom, a second file adds one film asserted to have both the Action and the Comedy genre. HermiT reports the ontology as inconsistent within seconds, and Protégé’s explanation (Figure 5) identifies the five axioms responsible: the disjointness of ActionMovie and ComedyMovie, the two hasValue definitions, and the two genre assertions. Under the open world assumption, nothing would make the two genres incompatible without the explicit disjointness axiom. The same axiom explains why only the primary genre is converted: with all genres, every action comedy would make the ontology inconsistent.

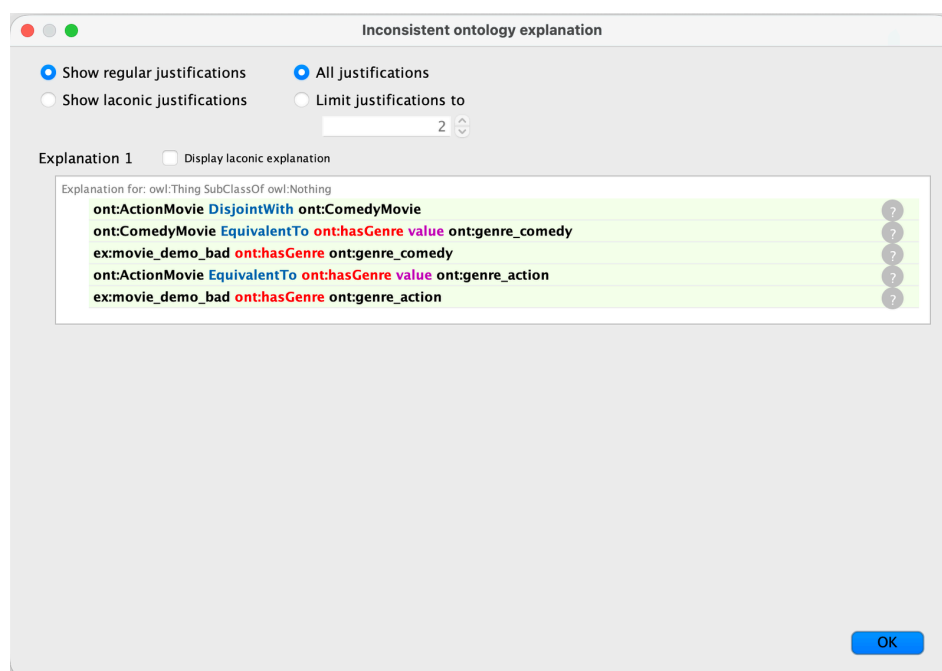


Figure 5: Protégé’s explanation of the inconsistency caused by a film with two disjoint genres.

8 Dataset Metadata

Following the W3C recommendations for publishing data on the Web, the dataset describes itself in `dataset_metadata.ttl`, generated by a script from the actual files: DCAT [13] for the catalogue, the dataset and one distribution per Turtle file; VOID [14] for statistics and two linksets (920 links to Wikidata, 912 to DBpedia); and PROV-O [15] for provenance, recording that the dataset was derived from the two Kaggle files by one pipeline activity that used Wikidata, DBpedia and our scripts. Licences are stated per part: MIT for the code, CC BY-NC 4.0 for our RDF, the TMDb terms for the upstream facts, CC0 for Wikidata links and CC BY-SA for DBpedia links.

9 Discussion and Limitations

Two lessons stand out. First, matching on shared identifiers is far more reliable than matching on names: the identifier-based method linked 99.6% of films to both hubs, whereas title-based matching found nothing. Second, `owl:sameAs` statements imported from a large source must be verified: redirect resources silently assert identity between different entities, and a small manual sample was enough to expose the problem.

The project has the following limitations:

- `Director` and `Actor` are modelled as subclasses although they are roles.
- Only the primary genre of each film is kept, to keep the disjoint genre classes consistent.
- The URIs use `example.org` and cannot be dereferenced on the Web.
- `characterName` is a literal: TMDb gives only a name, so different characters with the same name cannot be distinguished.
- The dataset is a snapshot of 250 films, and two resources remain unlinked because their identifiers are ambiguous.

10 Conclusion and Future Work

We built a five-star Linked Open Data set of 250 films: an ontology that reuses standard vocabularies and supports reasoning, RDF with stable URIs, 1,832 identifier-based links to Wikidata and DBpedia with a measured precision of 96.7–100%, and 21 SPARQL queries, ten of them federated. Future work includes modelling roles with a `hasRole` property, keeping all genres with non-disjoint genre classes, publishing the data with dereferenceable URIs behind a SPARQL endpoint such as Apache Jena Fuseki, and extending the dataset with more films and awards.

Team Contributions

Member	Contributions
Đỗ Tuấn Minh (leader)	Ontology design, defined classes and SWRL rule, reasoning checks, awards from Wikidata, terminal SPARQL interface, demo
Nguyễn Tiến Đức	Data collection and cleaning, CSV-to-RDF conversion, Wikidata and DBpedia linking and its redirect correction, link evaluation, statistics, slides
Hoàng Huy Chiến	Dataset metadata, federated queries, the CastRole model and its queries, report, demo video

References

- [1] T. Berners-Lee. *Linked Data — Design Issues*, W3C, 2006 (five-star scheme added 2010). <https://www.w3.org/DesignIssues/LinkedData.html>; see also <https://5stardata.info/>.
- [2] C. Bizer, T. Heath, T. Berners-Lee. Linked Data — The Story So Far. *International Journal on Semantic Web and Information Systems*, 5(3):1–22, 2009.
- [3] J. Lehmann et al. DBpedia — A large-scale, multilingual knowledge base extracted from Wikipedia. *Semantic Web*, 6(2):167–195, 2015.
- [4] D. Vrandečić, M. Krötzsch. Wikidata: a free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85, 2014.
- [5] M. Fernández-López, A. Gómez-Pérez, N. Juristo. METHONTOLOGY: From ontological art towards ontological engineering. *AAAI Spring Symposium on Ontological Engineering*, 1997.
- [6] M. Grüninger, M. S. Fox. Methodology for the design and evaluation of ontologies. *IJCAI Workshop on Basic Ontological Issues in Knowledge Sharing*, 1995.
- [7] N. Guarino, C. Welty. Evaluating ontological decisions with OntoClean. *Communications of the ACM*, 45(2):61–65, 2002.
- [8] I. Horrocks et al. SWRL: A Semantic Web Rule Language Combining OWL and RuleML. W3C Member Submission, 2004.
- [9] N. Noy, A. Rector (eds.). Defining N-ary Relations on the Semantic Web. W3C Working Group Note, 2006.
- [10] H. Halpin, P. J. Hayes, J. P. McCusker, D. L. McGuinness, H. S. Thompson. When owl:sameAs isn't the same: An analysis of identity in Linked Data. *ISWC*, 2010.
- [11] B. Glimm, I. Horrocks, B. Motik, G. Stoilos, Z. Wang. HermiT: An OWL 2 Reasoner. *Journal of Automated Reasoning*, 53(3):245–269, 2014.
- [12] M. A. Musen. The Protégé project: a look back and a look forward. *AI Matters*, 1(4):4–12, 2015.

- [13] W3C. Data Catalog Vocabulary (DCAT) — Version 3. W3C Recommendation, 2024.
- [14] K. Alexander, R. Cyganiak, M. Hausenblas, J. Zhao. Describing Linked Datasets with the VoID Vocabulary. W3C Interest Group Note, 2011.
- [15] T. Lebo, S. Sahoo, D. McGuinness (eds.). PROV-O: The PROV Ontology. W3C Recommendation, 2013.
- [16] The Movie Database (TMDb). TMDb 5000 Movie Dataset, Kaggle. <https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata>.